

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 1

Algoritmos Greedy

9 de abril de 2026

Ignacio Malmierca
113695

Nadeska Millan
110779

Samuel Herrera
112861

Introducción

En el presente trabajo práctico se aborda el diseño y análisis de algoritmos greedy, una técnica fundamental en la resolución de problemas de optimización. El objetivo principal es comprender en qué contextos este enfoque permite obtener soluciones óptimas y cómo justificar formalmente su correctitud.

Para ello, se propone el siguiente problema: se debe asistir a Scaloni en el análisis de los próximos n rivales de la selección. Cada rival requiere un procesamiento en dos etapas secuenciales: primero, un análisis por parte de Scaloni, que demanda un tiempo s_i , y luego una revisión por parte de un ayudante, que toma un tiempo a_i . Si bien existen suficientes ayudantes para trabajar en paralelo, cada uno puede comenzar únicamente una vez que Scaloni haya finalizado su parte correspondiente.

El desafío consiste en determinar un orden en el que Scaloni debe analizar los rivales de manera tal que el tiempo total necesario para completar todos los análisis sea mínimo.

A lo largo del trabajo, se realizará un análisis formal del problema, se propondrá un algoritmo de tipo greedy para su resolución y se demostrará su optimalidad. Asimismo, se estudiará la complejidad del algoritmo propuesto y, de manera opcional, se explorarán distintos escenarios y variaciones en los datos de entrada para evaluar su impacto en el desempeño y la optimalidad de la solución.

1. Análisis del problema y regla greedy

El problema planteado indica la necesidad de un doble análisis de cada rival: un primer análisis realizado por Scaloni y un segundo análisis realizado por un ayudante. Se cuenta con n ayudantes, es decir, la misma cantidad que rivales, por lo que la segunda etapa puede realizarse en paralelo.

Cada rival i requiere un tiempo s_i de análisis por parte de Scaloni y un tiempo a_i por parte de un ayudante. Si bien los ayudantes trabajan en paralelo, cada uno puede comenzar únicamente una vez que Scaloni haya finalizado el análisis del rival correspondiente.

En primer lugar, se observa que el tiempo total que le toma a Scaloni analizar todos los rivales es independiente del orden en que los procese. Dicho tiempo está dado por:

$$T_S = s_1 + s_2 + \cdots + s_n = \sum_{i=1}^n s_i = K$$

Dado que se trata de una suma de números naturales, se cumple la propiedad conmutativa, es decir:

$$s_1 + s_2 + \cdots + s_n = s_2 + s_1 + \cdots + s_n = K$$

Por lo tanto, el orden en el que Scaloni analiza los rivales no afecta su tiempo total de procesamiento. Sin embargo, este orden sí influye en el momento en que cada ayudante puede comenzar su tarea, y en consecuencia, en el tiempo total de finalización del proceso completo.

El objetivo del problema es minimizar el tiempo en que todos los análisis finalizan, lo cual está determinado por el instante en que termina el último ayudante.

Regla Greedy

Dado que el tiempo de Scaloni es fijo e independiente del orden, la estrategia óptima debe enfocarse en minimizar el impacto de los tiempos de los ayudantes. Para ello, se propone la siguiente regla greedy:

Ordenar los rivales en orden decreciente según el tiempo a_i , y procesarlos en ese orden.

La intuición detrás de esta estrategia es que los rivales cuyo análisis por parte del ayudante requiere más tiempo deben comenzar lo antes posible. De esta forma, dichos análisis se ejecutan en paralelo mientras Scaloni continúa procesando los restantes.

Si un rival con gran tiempo a_i se deja para el final, su ejecución retrasará innecesariamente el tiempo total de finalización, ya que su análisis comenzará más tarde y se sumará al final del proceso.

En consecuencia, ordenar los rivales de manera decreciente según a_i permite:

- Priorizar los análisis más costosos en la segunda etapa.
- Maximizar el paralelismo entre los ayudantes.
- Minimizar el tiempo total de finalización.

Por lo tanto, esta estrategia greedy resulta adecuada para resolver el problema de manera óptima.

2. Demostración de optimalidad

Para demostrar que el algoritmo greedy propuesto es óptimo, utilizaremos un argumento de intercambio.

El algoritmo consiste en ordenar los rivales en orden decreciente según a_i . Supongamos, con el objetivo de llegar a una contradicción, que existe un orden óptimo que no cumple esta propiedad. Entonces, existen dos rivales consecutivos i y j tales que:

$$a_i < a_j \quad \text{y} \quad i \text{ aparece antes que } j$$

Esto implica que hay una inversión respecto del orden propuesto.

Sea T el tiempo acumulado hasta antes de procesar estos dos rivales. Analizamos el tiempo total en ambos casos:

Orden original (con inversión): $i \rightarrow j$

- Finalización de i : $T + s_i + a_i$
- Finalización de j : $T + s_i + s_j + a_j$

Por lo tanto, el tiempo total es:

$$C_1 = \max(T + s_i + a_i, T + s_i + s_j + a_j)$$

Orden intercambiado: $j \rightarrow i$

- Finalización de j : $T + s_j + a_j$
- Finalización de i : $T + s_j + s_i + a_i$

El tiempo total es:

$$C_2 = \max(T + s_j + a_j, T + s_j + s_i + a_i)$$

Comparación

Dado que $a_i < a_j$, se cumple:

$$\begin{aligned} T + s_i + s_j + a_j &> T + s_j + a_j \\ T + s_i + s_j + a_j &> T + s_i + s_j + a_i \end{aligned}$$

De esto se deduce que:

$$C_1 \geq C_2$$

Es decir, al intercambiar los elementos i y j , el tiempo total no aumenta (y puede incluso disminuir).

Por lo tanto, cualquier solución con inversiones puede transformarse en otra sin inversiones sin empeorar el resultado. Aplicando sucesivos intercambios, se llega a un orden en el cual los valores a_i están ordenados de forma decreciente.

Esto demuestra que el algoritmo greedy propuesto produce siempre una solución óptima.

3. Algoritmo y complejidad

3.1. Algoritmo

El algoritmo implementado sigue la estrategia greedy de procesar los rivales en orden decreciente según el tiempo de análisis del ayudante a_i .

La implementación se divide en tres funciones principales, cuyos códigos se presentan a continuación.

Lectura y construcción de la lista de rivales

Esta función se encarga de leer el archivo de entrada y construir la lista de rivales, insertándolos en orden mediante la función auxiliar de ordenamiento.

```
1 def formatear_archivo_entrada():
2
3     if len(sys.argv) != 2:
4         sys.exit(1)
5
6     rivales = []
7     nombre_archivo = sys.argv[1]
8
9     with open(nombre_archivo, 'r') as f:
10
11         next(f)
12         for linea in f:
13             linea = linea.strip()
14             if linea == "":
15                 continue
16             s, a = map(int, linea.split(",")) #conversion directa a enteros
17             rival = [s, a]
18
19             ordenar_rivales(rivales, rival)
20     return rivales
```

Ordenamiento de rivales (regla greedy)

Esta función inserta cada rival en la posición correspondiente de manera que la lista quede ordenada en forma decreciente según a_i . De esta forma, implementa la regla greedy del algoritmo.

```
1 def ordenar_rivales(arr_rivales, rival):
2     i = 0
3     while i < len(arr_rivales) and arr_rivales[i][1] >= rival[1]:
4         i += 1
5     arr_rivales.insert(i, rival)
```

Cálculo del tiempo total

Esta función recorre la lista de rivales ya ordenada y calcula el tiempo total de finalización.

```
1 def analizar_rivales(rivales):
2     tiempo_scaloni = 0
3     tiempo_total = 0
4     for s, a in rivales:
5         tiempo_scaloni += s
6         fin_rival = tiempo_scaloni + a
7
8         if fin_rival > tiempo_total:
9             tiempo_total = fin_rival
10    return tiempo_total
```

3.2. Complejidad

La función `formatear_archivo_entrada` recorre el archivo línea por línea, procesando un total de n rivales. Dado que cada iteración realiza operaciones de costo constante, esta etapa tiene complejidad $O(n)$.

Por cada rival leído se ejecuta la función `ordenar_rivales`, la cual contiene un ciclo del tipo:

```
while i < len(arr_rivales) and arr_rivales[i][1] >= rival[1]
```

En el peor caso, este ciclo puede recorrer todos los elementos de la lista parcial, cuyo tamaño es k en cada inserción. Además, la operación de inserción también puede requerir desplazar elementos, lo que implica un costo adicional de $O(k)$. Por lo tanto, insertar un rival tiene costo $O(k)$.

Dado que este proceso se repite para los n rivales, el costo total de la inserción ordenada es:

$$O(1 + 2 + \dots + n) = O(n^2)$$

Finalmente, la función `analizar_rivales` recorre la lista una única vez para calcular el tiempo total, lo que implica un costo de $O(n)$.

Sumando todas las etapas anteriores, se obtiene:

$$O(n) + O(n^2) + O(n) = O(n^2)$$

Por lo tanto, la complejidad total del algoritmo es $O(n^2)$.

4. Impacto de la variabilidad de s_i y a_i en el tiempo

En esta sección se analiza cómo la variabilidad de los valores s_i (tiempos de Scaloni) y a_i (tiempos de los ayudantes) afecta el tiempo total del algoritmo.

Se proponen cuatro casos representativos.

Caso 1: $a_i \gg s_i$

En este caso, los tiempos de los ayudantes son considerablemente mayores que los de Scaloni. Se puede observar que el tiempo total queda determinado principalmente por los valores de a_i , alcanzándose el máximo en las primeras iteraciones.

Ejemplo:

$$s_i = [40, 30, 20, 60], \quad a_i = [100, 200, 500, 350]$$

Evolución:

- tiempo total: 520, tiempo Scaloni: 20
- tiempo total: 520, tiempo Scaloni: 80
- tiempo total: 520, tiempo Scaloni: 110
- tiempo total: 520, tiempo Scaloni: 150

Como se observa, el tiempo total queda determinado en la primera iteración, ya que el mayor a_i domina el resultado.

Caso 2: $a_i \approx s_i$

Cuando los valores de a_i y s_i son similares, el tiempo total depende tanto del acumulado de Scaloni como de los tiempos de los ayudantes.

Ejemplo:

$$s_i = [21, 38, 92, 100, 55, 27, 50, 41]$$

$$a_i = [17, 42, 96, 96, 55, 24, 43, 47]$$

Evolución:

- tiempo total: 188, tiempo Scaloni: 92
- tiempo total: 288, tiempo Scaloni: 192
- tiempo total: 306, tiempo Scaloni: 251
- tiempo total: 339, tiempo Scaloni: 292
- tiempo total: 385, tiempo Scaloni: 342
- tiempo total: 422, tiempo Scaloni: 380
- tiempo total: 431, tiempo Scaloni: 407
- tiempo total: 445, tiempo Scaloni: 428

En este caso, el tiempo total se actualiza en múltiples iteraciones, debido al crecimiento acumulativo de s_i .

Caso 3: $s_i \gg a_i$

Cuando los tiempos de Scaloni son mucho mayores que los de los ayudantes, el tiempo total está dominado por la suma acumulada de s_i .

Ejemplo:

$$s_i = [1000, 400, 300, 1500, 600, 700, 100, 630, 22900]$$

$$a_i = [100, 120, 78, 250, 150, 200, 20, 47, 2900]$$

Evolución (resumida):

- tiempo total inicial: 25800
- ...
- tiempo total final: 28150

En este escenario, el tiempo total se alcanza en las últimas iteraciones, ya que depende principalmente del crecimiento acumulado del tiempo de Scaloni.

Caso 4: $s_i = a_i$

Cuando los valores de s_i y a_i son iguales, el comportamiento es similar al caso en que ambos son comparables.

Ejemplo:

$$s_i = [1000, 1000, 700, 100, 20, 5]$$

$$a_i = [1000, 1000, 700, 100, 20, 5]$$

Evolución:

- tiempo total: 2000
- tiempo total: 3000
- tiempo total: 3400
- tiempo total se mantiene en 3400

En este caso, el máximo no depende de una iteración específica, sino de la distribución general de los valores.

La variabilidad de los valores s_i y a_i afecta el momento en que se alcanza el tiempo total máximo, pero no modifica la estrategia óptima del algoritmo.

5. Impacto de la variabilidad de s_i y a_i en la optimalidad

En esta sección se analiza si la variabilidad de los valores s_i (tiempos de Scaloni) y a_i (tiempos de los ayudantes) afecta la optimalidad del algoritmo greedy propuesto.

El algoritmo se basa en ordenar los rivales en forma decreciente según a_i . Esta estrategia asegura que los análisis que más tiempo requieren por parte de los ayudantes comiencen lo antes posible, minimizando el tiempo total de finalización.

A continuación se analizan distintos casos utilizando ejemplos concretos.

Caso 1: $a_i \gg s_i$

$$s_i = [40, 30, 20, 60], \quad a_i = [100, 200, 500, 350]$$

En este caso, los valores de a_i dominan claramente. El algoritmo greedy prioriza correctamente los mayores a_i , logrando que los análisis más costosos comiencen primero.

Si no se respetara este orden (por ejemplo, procesando primero un a_i pequeño), el tiempo total aumentaría, ya que el mayor a_i quedaría retrasado.

Por lo tanto, la variabilidad en este caso no afecta la optimalidad, sino que refuerza la necesidad del criterio greedy.

Caso 2: $a_i \approx s_i$

$$s_i = [21, 38, 92, 100, 55, 27, 50, 41]$$

$$a_i = [17, 42, 96, 96, 55, 24, 43, 47]$$

Cuando los valores son similares, el impacto de cada decisión es más equilibrado. Sin embargo, el orden decreciente según a_i sigue siendo óptimo.

Cualquier intercambio que viole este orden (por ejemplo, colocar un a_i menor antes que uno mayor) puede incrementar el tiempo total, como se demuestra mediante el argumento de intercambio.

Por lo tanto, la variabilidad baja tampoco afecta la optimalidad del algoritmo.

Caso 3: $s_i \gg a_i$

$$s_i = [1000, 400, 300, 1500, 600, 700, 100, 630, 22900]$$

$$a_i = [100, 120, 78, 250, 150, 200, 20, 47, 2900]$$

En este caso, los valores de s_i dominan el tiempo total. Sin embargo, esto no modifica el criterio de ordenamiento óptimo.

El algoritmo continúa ordenando por a_i , lo cual sigue siendo correcto, ya que el tiempo total depende del máximo de las sumas acumuladas más a_i .

Cambiar el orden en función de s_i no garantiza una mejora y puede empeorar el resultado.

Por lo tanto, aunque s_i tenga mayor impacto en el valor final, no afecta la optimalidad del criterio greedy.

Caso 4: $s_i = a_i$

$$s_i = [1000, 1000, 700, 100, 20, 5]$$

$$a_i = [1000, 1000, 700, 100, 20, 5]$$

Cuando s_i y a_i son iguales, el problema se vuelve simétrico. En este contexto, el orden decreciente según a_i coincide con el orden según s_i .

El algoritmo greedy sigue produciendo una solución óptima, y cualquier permutación que respete este orden mantiene la optimalidad.

A partir de los casos analizados, se concluye que la variabilidad de los valores s_i y a_i no afecta la optimalidad del algoritmo greedy.

6. Ejemplos de ejecución y validación

En esta sección se presentan distintos ejemplos de ejecución del algoritmo con el objetivo de validar su correctitud y corroborar los resultados obtenidos en el análisis teórico.

Los casos de prueba fueron definidos en archivos de entrada, los cuales representan distintos escenarios según la relación entre los valores s_i y a_i .

6.1. Casos analizados

Se analizaron los siguientes escenarios:

- $a_i \gg s_i$ (archivo: `ayudantes_mayor.txt`)
- $a_i \approx s_i$ (archivo: `tiempos_similares.txt`)
- $s_i \gg a_i$ (archivo: `scaloni_mayor.txt`)
- $s_i = a_i$ (archivo: `tiempos_iguales.txt`)

6.2. Resultados obtenidos

A continuación se presentan los resultados obtenidos en cada caso:

Caso 1: $a_i \gg s_i$

- Tiempo total: 18850
- Tiempo de Scaloni acumulado: 40

- Tiempo de ayudante dominante: 18810 (video 25)
- Cantidad de rivales: $n = 100$

En este caso, el tiempo total está claramente dominado por el mayor valor de a_i , lo cual coincide con el análisis teórico.

Caso 2: $a_i \approx s_i$

- Tiempo total: 5882
- Suma total de s_i : 5865
- Tiempo de ayudante relevante: 17 (video 7)
- Cantidad de rivales: $n = 100$

Aquí, el tiempo total depende tanto de la suma acumulada de s_i como de los valores de a_i , lo cual confirma el comportamiento esperado.

Caso 3: $s_i \gg a_i$

- Tiempo total: 792397532
- Suma total de s_i : 792387393
- Tiempo de ayudante relevante: 10139 (video 20)
- Cantidad de rivales: $n = 255$

En este caso, el tiempo total está dominado por la suma de los tiempos de Scaloni, lo cual coincide con el análisis previo.

Caso 4: $s_i = a_i$

- Tiempo total: 5288
- Suma total de s_i : 5286
- Tiempo de ayudante relevante: 2 (video 11)
- Cantidad de rivales: $n = 100$

Cuando los valores son iguales, el tiempo total depende de la distribución general de los datos, validando el comportamiento analizado.

7. Conclusiones

Acá irían las conclusiones de todo nuestro trabajo :)